DeltaMint - Information Security Policy

> Written to satisfy the seven areas Alpaca's OAuth due-diligence questionnaire lists as the minimum: data classification and handling, access control and privileged access management, encryption at rest and in transit, vulnerability and patch management, incident response and disaster recovery, physical security, and vendor risk management.
>
> **Workstation security below:** disk encryption and MFA are confirmed true. Auto-updates and a password manager were not confirmed, so neither is claimed - the [!] stays on those two until confirmed or made true. Overstating security posture to a broker is worse than admitting a gap.

Owner: Osama Maghawry, CEO, Optvest Inc.

Contact: support@deltamint.app

Version: 1.1 - August 29, 2026

Review cycle: annually, or after any material change to the architecture or any security incident.

1. Scope and architecture

DeltaMint is a serverless web application. There are no company-operated servers, virtual machines, or network appliances. The estate is:

Layer	Provider	Contents
Static frontend	Cloudflare Workers	Browser bundle only; no secrets, no user data
API / business logic	Supabase Edge Functions (managed Deno)	Server-side logic, secret material in environment
Database, auth	Supabase (managed PostgreSQL)	User records, brokerage credentials (encrypted), cached market reference data
Source control, CI/CD, backups	GitHub	Application source, deployment workflows, and the nightly database backup artefacts (§6)
Brokerage	User's own broker, via API	Order routing and account data

This policy covers all of the above, the founder's workstation, and the third-party services listed in §8.

2. Data classification and handling

Four classes, with handling rules attached:

Class 1 - Secret. Brokerage API keys and secrets, OAuth access tokens, the credential encryption key, the database service-role key, and the database connection string held as a CI secret for the backup job (§6). Never displayed in the user interface, never logged, never transmitted to the browser, never committed to source control. Encrypted at rest (§4). Held in plaintext only in edge function memory, for the duration of a single request.

Class 2 - Confidential. User account records, positions, orders, trade history, and account balances retrieved from a user's broker, together with the account number of each connected brokerage account. Scoped to the owning user by row-level security; never shared between users; never sold or disclosed to third parties for marketing.

Class 2 also covers internal support records - free-text notes and a status/tag record an administrator may keep about a user's account for support and customer-relationship purposes. These are personal data about the user, so they are named here rather than treated as ordinary internal material. They live in tables with row-level security enabled and **no policies at all**, which denies every client role outright: no browser session can read them under any circumstances, and the only access path is a server-side function that re-verifies the administrator role. They are deleted when the user's account is deleted. The Privacy Policy discloses that such records are kept.

Class 3 - Internal. Application source code, deployment configuration, system logs. Access limited to authorised personnel (§3).

Class 4 - Public. Marketing site content, published legal pages, cached market reference data such as scheduled earnings dates, which is public information identical for every user.

Retention: Class 2 data persists while an account is active. On account deletion or brokerage disconnection, stored credentials are deleted from the live database; copies inside nightly backup artefacts age out on the 30-day cycle in §6, which is the true outer bound on any deletion. Logs are retained on the managed platforms under their standard retention and contain no Class 1 material.

3. Access control and privileged access management

- **Users** authenticate to DeltaMint by email and password through Supabase Auth. Passwords are salted and hashed by the platform; we never see them.
- **Authorisation** is enforced in two independent layers. PostgreSQL row-level security scopes every user-facing table to `auth.uid()`, so one user's session cannot read another user's rows. Independently, every server-side function verifies the caller's JWT and re-confirms ownership of the specific brokerage account before acting on it - the service-role client bypasses row-level security, so this ownership check is what actually enforces scoping there.
- **Credential columns are revoked from the browser-facing database role entirely**, verified against the live project. A client session cannot read stored brokerage credentials even if application code asked it to.
- **Administrative access** is held by the founder alone and is granted two ways: an email allowlist held as an environment secret outside the database - the bootstrap, which nothing inside the application can add itself to - or an `admin` role on a user profile, which only an existing administrator can grant. Both are checked **server-side in a single shared function**, so the rule exists in one place and cannot drift between endpoints. The browser also asks the server whether it is an administrator rather than deciding for itself; what it renders is presentation, never the boundary.
- **What an administrator can do**, stated in full rather than by example: run the maintenance jobs that re-encrypt credentials or refresh reference data; read account-level records across all users (email address, sign-up and last-sign-in times, number of connected accounts, number of trades, realised profit and loss); write the internal support records in §2; publish marketing content; and change the operator settings below. An administrator **cannot** read any user's brokerage credentials - those are encrypted under a key held outside the database (§4) - and no administrative path places, modifies, or cancels an order in a user's account.

- **Brokerage connections are OAuth-only for users.** There is no customer-facing path for entering a brokerage API key and secret: a user connects by authorising DeltaMint through their broker's own consent screen. Manual key entry survives solely as an internal testing capability for the administrator, held **off by default** in a setting the browser cannot read, and enforced in the server-side function that writes credentials - a request carrying a key and secret is refused unless the caller is an administrator *and* the setting is on. A non-administrator is refused either way.
- **Privileged platform access** (GitHub, Cloudflare, Supabase, domain registrar, broker) is limited to the founder as the sole operator, each protected by multi-factor authentication [!]. There are no shared accounts and no shared passwords. Access is reviewed whenever personnel change; with a sole operator, that means at incorporation and at each hire.
- **Least privilege:** the frontend holds only the publishable anon key, which is powerless without a valid user session and is guarded by row-level security. The service-role key exists only in the edge function environment.

4. Encryption of data at rest and in transit

In transit. TLS for all traffic, end to end: browser to Cloudflare, browser to Supabase, edge functions to the broker's API. HSTS is set on the public domain. No plaintext HTTP endpoint is served.

At rest. Managed platform storage is encrypted at rest by the provider. On top of that, brokerage credentials receive application-layer encryption before they are stored:

- **AES-256-GCM**, a fresh 96-bit initialisation vector per value, stored as `v1:<iv>:<ciphertext>`, base64-encoded.
- GCM is authenticated encryption, so tampering with stored ciphertext is detected on decryption rather than silently accepted.
- **The key is held as an edge function environment secret and is never present in the database.** This is the point of encrypting at the application layer rather than relying on disk encryption alone: a database dump, a leaked service-role key, or SQL injection yields ciphertext only. An attacker must compromise two independent systems, not one.
- **Key rotation is supported without user involvement.** A previous-key secret allows values written under the outgoing key to be decrypted while an administrative job re-encrypts them under the new key. Rotation therefore requires no user to re-enter credentials and causes no outage.

5. Vulnerability and patch management

- **No self-managed servers or operating systems exist to patch.** The runtime, database engine, and TLS termination are managed by Supabase and Cloudflare and are patched by those providers under their own programmes.
- **Dependencies** are pinned in version control. GitHub Dependabot alerts are enabled on the repository; security advisories affecting a dependency are triaged on receipt, and critical vulnerabilities in a reachable code path are patched and deployed the same day.
- **Static analysis** runs in continuous integration: linting gates every change to server-side code, and a deployment fails rather than shipping a change that does not pass. Every change must additionally produce a clean production build before it can be served.
- **Platform security advisories** from Supabase are reviewed and acted on.

- **Deployment** is automated from version control. Every production change is an auditable, attributable commit; there is no manual editing of running code.

6. Incident response and disaster recovery

Owner. Osama Maghawry is accountable for detecting, triaging, and responding to security incidents. Reports may be sent to support@deltamint.app.

Response procedure.

1. **Detect and triage** - classify severity by whether Class 1 or Class 2 data is implicated.
2. **Contain** - for suspected credential exposure: rotate the credential encryption key, revoke the affected platform keys, and invalidate user sessions. Where brokerage credentials may be affected, disconnect the affected accounts so no order can be routed with a suspect credential.
3. **Eradicate and recover** - patch the cause, redeploy from a known-good commit, and verify.
4. **Notify** - affected users, and the broker where their systems or customer accounts are implicated, without undue delay. Regulatory notification as applicable.
5. **Review** - a written post-incident record of cause, impact, and the change made to prevent recurrence.

Disaster recovery. Application code is fully reproducible from version control; a total loss of the hosting account is recoverable by redeploying from the repository.

Database backups are our own, not the platform's. The managed database tier in use provides neither scheduled backups nor point-in-time recovery, so rather than rely on one, DeltaMint runs its own:

- A **full logical dump of the production database every night at 08:00 UTC**, compressed and stored as a private build artefact, encrypted at rest by the platform that holds it, reachable only by the founder's MFA-protected account.
- **30 days of dumps are retained**, giving 30 daily recovery points rather than a single latest copy - which is what allows recovery from a fault discovered late, not merely from a total loss.
- The job is **defined in version control alongside the application**, so the schedule, retention, and restore command are auditable and change only by reviewed commit. The restore procedure is one documented command against a fresh database.
- Backups carry Class 1 and Class 2 data - the credential columns inside them remain encrypted under the application-layer key of §4, which is **not** in the database and therefore not in the dump. A leaked backup yields ciphertext for every credential it contains.

Because no state lives on a workstation, loss of the founder's machine causes no data loss and no service interruption.

7. Physical security

No offices, data centres, or company-owned server hardware exist. Physical security of the production estate is inherited from Cloudflare and Supabase and their underlying providers, whose data centres operate under SOC 2 controls.

Workstation security. The only company-controlled physical asset is the founder's workstation. It is protected by:

- Full-disk encryption (FileVault / BitLocker). **Confirmed enabled.**

- [!] Automatic operating system and browser updates - not yet confirmed.
- [!] A password manager with unique credentials per service - not yet confirmed.
- Multi-factor authentication on every privileged account (§3). **Confirmed enabled on GitHub, Cloudflare, Supabase, and the broker.**
- Screen lock on idle, and the device is not shared.

No production secret is stored on the workstation in plaintext; secrets live in the managed platforms' secret stores and are referenced, not copied.

8. Vendor risk management

Third parties are limited to those necessary to run the Service. Each is a established provider selected in part for its published security posture, and each is reviewed annually or on material change.

Vendor	Purpose	Data exposure
Supabase	Database, authentication, serverless functions	Class 1 and 2
Cloudflare	Static hosting, DNS, TLS, DDoS protection	Class 4 only
GitHub	Source control, CI/CD, and storage of the nightly database backups (§6)	Class 3, plus Class 1 and 2 inside backup artefacts - credentials within them stay encrypted under a key held elsewhere
Broker (user-authorised)	Order routing, account and market data	Class 1 and 2, at the user's direction
Market reference data provider	Scheduled earnings dates	Class 4 only; no user data sent
Payment processor	Subscription billing	Card data handled entirely by the processor; we never receive or store card numbers

We do not sell user data, and we do not share it with any party other than those above for the purposes listed.

The Privacy Policy discloses these sub-processors **by category** - database and authentication, hosting, source control and backup storage, payment processing - rather than by name. Naming each provider on a public page tells an attacker which vendor accounts to target for no benefit to the reader, so the named list is this table: current, maintained here, and provided to a counterparty, customer, or regulator on request.

9. Endpoint protection

(This section answers the questionnaire's endpoint-protection question directly.)

Production. There are no company-operated servers or workstations in the production path, so there is no endpoint on which to install anti-malware software. Production code executes in provider-managed, sandboxed, ephemeral runtimes - Cloudflare Workers isolates and Supabase Edge Function isolates - which are rebuilt from an immutable deployment artefact on each release. The attack surface is reduced structurally rather than by scanning: no persistent host, no shell access, no long-lived process, and no capacity to install software into a running environment. Malicious code cannot be introduced except through the source repository, which is

protected by authenticated, multi-factor-gated access and by static analysis and lint gates in continuous integration.

Corporate. The single corporate endpoint is the founder's workstation, protected as described in §7 - full-disk encryption enabled, and no production data stored locally. Administrative access to every production platform - GitHub, Cloudflare, Supabase, and the broker - requires multi-factor authentication, so compromise of the workstation alone does not yield production access.